DRIVEIQ · SOFTWARE REQUIREMENTS SPECIFICATION

SRS	Document Type
v1.0	Version
InsurTech	Domain
Final	Status

USAGE-BASED VEHICLE INSURANCE

DriveIQ

Software Requirements Specification (SRS)

Version 1.0 · March 2026 · IEEE 830 Standard

TECHNOLOGY STACK

Spring Boot 3 · Angular 17 · JWT Auth · PostgreSQL · Spring JPA · Payment Sim

DOCUMENT INFORMATION

Document Title	DriveIQ UBI Platform — Software Requirements Specification
Version	1.0
Date	March 17, 2026
Domain	InsurTech / FinTech
Status	Final
Standard	IEEE 830-1998

PREPARED BY
Bhavana Sri Tejavath

Table of Contents

1. Introduction 3
- 1.1 Purpose 3
- 1.2 Scope 3
- 1.3 Definitions and Acronyms 3
- 1.4 Document Conventions 4
- 1.5 References 4

2. Overall System Description 5
- 2.1 Product Perspective 5
- 2.2 System Context Diagram 5
- 2.3 User Classes and Characteristics 5
- 2.4 Operating Environment 6
- 2.5 Assumptions and Dependencies 6

3. Functional Requirements 7
- 3.1 Authentication & User Management 7
  - 3.1.1 User Registration (FR-001) 7
  - 3.1.2 User Login (FR-002) 7
  - 3.1.3 Password Reset (FR-003) 7
  - 3.1.4 User Profile Management (FR-004) 7
  - 3.1.5 Admin User Management (FR-005) 7
- 3.2 Policy Management 7
  - 3.2.1 Policy Catalog (FR-006) 7
  - 3.2.2 Policy Application / Order Submission (FR-007) 8
  - 3.2.3 Order Status Lifecycle (FR-008) 8
- 3.3 Underwriter Workflow 8
  - 3.3.1 Application Review (FR-009) 8
  - 3.3.2 Risk Scoring (FR-010) 8
  - 3.3.3 Underwriter Decision Actions (FR-011) 8
- 3.4 Subscription Management 9
  - 3.4.1 Subscription Creation (FR-012) 9
  - 3.4.2 Subscription Lifecycle (FR-013) 9
- 3.5 Vehicle Management 9
  - 3.5.1 Vehicle Registration (FR-014) 9
  - 3.5.2 Vehicle-Subscription Linking (FR-015) 9
- 3.6 Usage Data Tracking 9
  - 3.6.1 Monthly Usage Submission (FR-016) 9
  - 3.6.2 Usage History (FR-017) 10
- 3.7 Premium Calculation Engine 10
  - 3.7.1 Rule-Based Premium Engine (FR-018) 10
  - 3.7.2 Premium Rule Administration (FR-019) 10
  - 3.7.3 Premium History (FR-020) 10
- 3.8 Claims Management 10
  - 3.8.1 Claim Submission (FR-021) 10
  - 3.8.2 Claim Review (FR-022) 11
  - 3.8.3 Claim Concurrency Control (FR-023) 11
- 3.9 Payment Processing 11
  - 3.9.1 Payment Initiation (FR-024) 11
  - 3.9.2 Payment Verification (FR-025) 11
  - 3.9.3 Payment History (FR-026) 11
- 3.10 Notifications 12
  - 3.10.1 In-App Notifications (FR-027) 12
  - 3.10.2 Email Notifications (FR-028) 12
- 3.11 Dashboard & Reporting 12
  - 3.11.1 Admin Dashboard (FR-029) 12
  - 3.11.2 Underwriter Dashboard (FR-030) 12

4. Data Requirements & Database Design 13
- 4.1 Entity Relationship Overview 13
- 4.2 Core Entity Definitions 13
  - 4.2.1 Users Table 13
  - 4.2.2 PolicyOrder Table 13
  - 4.2.3 UsageData Table 14
  - 4.2.4 PremiumRule Table 14
- 4.3 Document Storage Strategy 15

5. REST API Specification 16
- 5.1 API Standards 16
  - 5.1.1 Standard Pagination Response 16
- 5.2 Authentication API 16
- 5.3 Core Resource APIs 16
- 5.4 Payment Flow - Sequence 17

6. Security Requirements 19
- 6.1 Authentication & Authorization 19
  - 6.1.1 JWT-Based Authentication (NFR-001) 19
  - 6.1.2 Role-Based Access Control (NFR-002) 19
  - 6.1.3 Password Security (NFR-003) 19
- 6.2 Data Security 19
  - 6.2.1 Payment Security (NFR-004) 19
  - 6.2.2 Input Validation (NFR-005) 19
- 6.3 Controller Security Reference 19

7. Non-Functional Requirements 21
- 7.1 Performance Requirements 21
- 7.2 Scalability Requirements 21
- 7.3 Reliability Requirements 21
- 7.4 Usability Requirements 21
- 7.5 Maintainability Requirements 22
- 7.6 Security Compliance Requirements 22

8. System Architecture & Technology Stack 23
- 8.1 Backend Technology Stack 23
- 8.2 Frontend Technology Stack 23
- 8.3 Frontend Module Structure 23
- 8.4 Angular Route Protection 24

9. Business Rules & State Machines 25
- 9.1 Core Business Rules 25
- 9.2 State Machine Diagrams 25
  - 9.2.1 PolicyOrder Status Flow 25
  - 9.2.2 Claim Status Flow 26
  - 9.2.3 PolicySubscription Status Flow 26
  - 9.2.4 Payment Status Flow 26

10. End-to-End Workflows 28
- 10.1 Policy Application to Active Subscription 28
- 10.2 Claim Submission to Settlement 28
- 10.3 Monthly Usage and Premium Calculation 28

11. Error Handling & API Error Reference 30
- 11.1 Standard HTTP Error Codes 30
- 11.2 Error Response Format 30
- 11.3 Concurrency Control Implementation 30

12. Configuration & Deployment 32
- 12.1 Environment Configuration 32
- 12.2 Deployment Prerequisites 32
- 12.3 Build & Run Commands 32

13. Data Transfer Object (DTO) Reference 34
- 13.1 Authentication DTOs 34
- 13.2 Domain DTOs 34

14. Appendices 36
- Appendix A: Premium Rule Engine - Strategy Pattern 36
- Appendix B: Risk Score Computation 36
- Appendix C: Sample cURL API Commands 37
- Appendix D: Notification Type Quick Reference 37
- Appendix E: Revision History 38

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the DriveIQ Usage-Based Insurance (UBI) Platform. It is intended to serve as the primary reference for developers, quality assurance engineers, project managers, and stakeholders involved in the design, development, testing, and maintenance of the system.

The document covers all system components including the Angular-based frontend, the Spring Boot backend, database design, REST API contracts, security architecture, business rules, and integration points with third-party services such as Razorpay and SMTP email providers.

1.2 Scope

DriveIQ is a modern web-based insurance platform operating under the tagline "Pay as you drive." The platform dynamically adjusts motor vehicle insurance premiums based on a customer's actual driving behavior — including total distance driven, night driving hours, and risk classification — enabling fairer pricing and rewarding safe drivers with lower premiums.

The system encompasses the following key functional areas:

- Customer registration, authentication, and profile management
- Policy browsing, application submission, and document upload
- Underwriter-driven application review with risk scoring
- Premium calculation using a configurable rule engine
- Monthly driving usage data submission and tracking
- Claims submission, review, and settlement workflows
- Integrated online payment processing via Razorpay
- Administrative dashboard with analytics and reporting
- Role-based access control for four distinct user roles
- Automated email and in-app notifications for system events

1.3 Definitions and Acronyms

Term / Acronym	Definition
UBI	Usage-Based Insurance — insurance where premiums are calculated based on actual usage patterns
SRS	Software Requirements Specification — this document
JWT	JSON Web Token — a compact, URL-safe token standard for authentication
RBAC	Role-Based Access Control — restricting access based on user roles
API	Application Programming Interface

Term / Acronym	Definition
REST	Representational State Transfer — architectural style for distributed systems
DTO	Data Transfer Object — objects that carry data between processes
HMAC	Hash-based Message Authentication Code — used for payment signature verification
BCrypt	Password hashing algorithm used for secure credential storage
JPA	Java Persistence API — ORM specification for Java
SPA	Single Page Application — Angular-based frontend
SMTP	Simple Mail Transfer Protocol — email communication standard
DXA	Document eXtended Attribute unit — 1/1440th of an inch in Word documents
INR	Indian Rupee — currency used in payment transactions

1.4 Document Conventions

This document follows IEEE 830 SRS conventions. Functional requirements are labeled FR-XXX. Non-functional requirements are labeled NFR-XXX. Each requirement section is organized by subsystem.

1.5 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Spring Boot 3.x Documentation — https://spring.io/projects/spring-boot
- Angular Documentation — https://angular.io/docs
- Razorpay Payment Gateway API Reference — https://razorpay.com/docs/api
- JWT RFC 7519 — https://tools.ietf.org/html/rfc7519
- PostgreSQL 15 Official Documentation — https://www.postgresql.org/docs/

2. Overall System Description

2.1 Product Perspective

DriveIQ is a standalone, full-stack web application with no dependency on legacy insurance systems. It is designed as a cloud-ready, stateless application that can be deployed independently on any modern server infrastructure. The system integrates with two external services: Razorpay for payment processing and an SMTP server (Gmail) for email delivery.

> *DriveIQ distinguishes itself from traditional flat-rate insurance by introducing a real-time, rule-driven premium engine that factors in driving behavior data, thereby incentivizing safe driving practices.*

2.2 System Context Diagram

The high-level system architecture positions the Angular frontend as the primary user interface, communicating with the Spring Boot backend over HTTP/REST (JSON). The backend orchestrates all business logic, data persistence, and external service integrations:

```
┌──────────────────────────────────────────────────────────────────────┐
│                    DRIVEIQ SYSTEM CONTEXT                            │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   [Customer]  [Admin]  [Underwriter]  [Claims Officer]               │
│        │         │           │               │                      │
│        └─────────┴───────────┴───────────────┘                      │
│                          │                                           │
│                  Angular SPA (:4200)                                 │
│                  TypeScript + Tailwind CSS                           │
│                          │                                           │
│                      HTTP/REST JSON                                  │
│                          │                                           │
│              Spring Boot Backend (:8080)                             │
│         ┌──────────────────────────────┐                            │
│         │  JWT Auth → Controllers      │                            │
│         │  Service Layer (Business Logic)  │                        │
│         │  JPA Repository Layer        │                            │
│         └──────────┬──────────────────┘                             │
│                    │              │                                  │
│            PostgreSQL DB    Razorpay + SMTP                          │
└──────────────────────────────────────────────────────────────────────┘
```

2.3 User Classes and Characteristics

Role	Access Area	Primary Responsibilities	Technical Proficiency
CUSTOMER	/customer/*	Browse policies, submit applications, upload documents, pay premiums, track usage, file claims	General public — low to medium
ADMIN	/admin/*	Full system oversight — manage users, policies, premium rules, view analytics and revenue	Internal staff — high
UNDERWRITER	/underwriter/*	Review policy applications, assess risk scores, approve/reject/request documents	Insurance domain expert — medium
CLAIMS_OFFICER	/claims/*	Review submitted claims, approve or reject, notify customers	Insurance domain expert — medium

2.4 Operating Environment

- Backend: Java 17+ runtime on any JVM-compatible OS (Linux/Windows/macOS)
- Frontend: Modern web browsers (Chrome 100+, Firefox 100+, Safari 15+, Edge 100+)
- Database: PostgreSQL 14+ with JPA/Hibernate ORM
- Network: HTTPS in production; HTTP locally on ports 4200 (frontend) and 8080 (backend)
- Email: Gmail SMTP (port 587, TLS) for notification delivery
- Payments: Razorpay payment gateway — INR currency only

2.5 Assumptions and Dependencies

1. A valid Razorpay merchant account is configured with test/live API keys.
2. A Gmail account is configured with App Password for SMTP relay.
3. PostgreSQL 14+ is available and the database 'driveiq' has been created.
4. Java 17+ is installed on the deployment server.
5. Node.js 18+ and Angular CLI are installed for frontend development.
6. JWT tokens are 24 hours long — users must re-authenticate after expiry.
7. Usage data submission is manual (customer-reported); GPS/telematics integration is out of scope for this version.

3. Functional Requirements

3.1 Authentication & User Management

3.1.1 User Registration (FR-001)

- The system shall allow new customers to self-register via a public registration endpoint.
- Registration requires: full name, email address, password, phone number, and age.
- Email addresses must be unique in the system.
- Passwords must be BCrypt-hashed before storage.
- On successful registration, the system shall send a welcome email to the customer.
- All ADMIN-role users shall receive an in-app notification (NEW_REGISTRATION) when a new customer registers.

3.1.2 User Login (FR-002)

- The system shall authenticate users via email and password credentials.
- On successful authentication, the server shall issue a signed JWT token (RS256 algorithm, 24-hour TTL).
- The JWT payload shall include the user's email, role, and user ID.
- Failed authentication attempts shall return HTTP 401.

3.1.3 Password Reset (FR-003)

- The system shall support a self-service password reset flow.
- A reset token shall be generated with a 15-minute expiry window.
- The token shall be emailed to the registered email address.
- A valid token and new password are required to complete the reset.

3.1.4 User Profile Management (FR-004)

- All authenticated users may view their own profile.
- Users may update their full name, phone number, and address.
- Email address is immutable and cannot be changed after registration.
- The system reads the user identity from the JWT security context, not a user-supplied parameter.

3.1.5 Admin User Management (FR-005)

- Admins may list, search, and filter all users.
- Admins may view any individual user's profile.
- Admins may reassign user roles by specifying a role ID.
- Admins may deactivate a user account.
- Staff accounts created by Admin trigger a STAFF_CREATED email and in-app notification.

3.2 Policy Management

3.2.1 Policy Catalog (FR-006)

- The system shall maintain a catalog of insurance policy templates.
- Each policy has: name, coverage type (COMPREHENSIVE or THIRD_PARTY), base premium, description, and active status.
- Policies are readable by ADMIN, UNDERWRITER, and CUSTOMER roles.
- Only ADMIN may create, update, or toggle the active status of a policy.
- Policy listings support pagination and search filtering.

3.2.2 Policy Application / Order Submission (FR-007)

- A CUSTOMER may submit a policy application (order) by selecting a policy and optionally linking a vehicle.
- Policy orders support document attachments (PDF, JPG, JPEG, PNG; max 5 MB each).
- Submitted documents are stored on the server filesystem under: ./data/policy-documents/{orderId}/
- Orders are created in PENDING status by default.
- The system generates a unique UUID-prefixed filename for each uploaded document.

3.2.3 Order Status Lifecycle (FR-008)

Policy orders follow a defined state machine:

```
PENDING ──────► APPROVED
   │
   ├──────────► REJECTED
   │
   └──────────► DOCUMENTS_REQUESTED ──► (Customer re-submits) ──► PENDING
```

3.3 Underwriter Workflow

3.3.1 Application Review (FR-009)

- Underwriters have access to a paginated list of all policy applications.
- The application view includes customer details, vehicle information, uploaded documents, risk score, and risk level.
- Underwriters may filter by application status: PENDING, APPROVED, REJECTED, DOCUMENTS_REQUESTED.
- Underwriters may download and preview submitted documents.

3.3.2 Risk Scoring (FR-010)

- Risk is computed dynamically for each pending application based on: driver age, vehicle type, vehicle age, and claims history.
- Risk levels are classified as LOW, MEDIUM, or HIGH.
- Risk score and level are locked (persisted) at the time of approval or rejection for audit trail purposes.

3.3.3 Underwriter Decision Actions (FR-011)

- Approve: Sets order status to APPROVED; creates a 1-year PolicySubscription; links vehicle if applicable; notifies customer via email and in-app notification.
- Reject: Sets order status to REJECTED; sends rejection email with underwriter remarks.
- Request Documents: Sets status to DOCUMENTS_REQUESTED; sends email specifying required documents.

3.4 Subscription Management

3.4.1 Subscription Creation (FR-012)

- A PolicySubscription is automatically created when an underwriter approves a policy order.
- Subscriptions have a 1-year validity: startDate = approval date, endDate = startDate + 365 days.
- Subscriptions are linked to the policy order and, optionally, to one or more vehicles.

3.4.2 Subscription Lifecycle (FR-013)

Subscription statuses follow this lifecycle:

```
ACTIVE ──▶ EXPIRED
ACTIVE ──▶ CANCELLED
EXPIRED ──▶ ACTIVE  (on renewal)
```

- Customers may view their own subscriptions.
- Admins may update subscription status and initiate renewals.
- Subscription renewal extends the end date by one additional year.

3.5 Vehicle Management

3.5.1 Vehicle Registration (FR-014)

- Customers may register vehicles with: vehicle number, vehicle type (CAR, BIKE, SUV, TRUCK, SPORTS), vehicle age, and registration date.
- Vehicle numbers must be unique in the system.
- Customers may update or delete their registered vehicles.
- Admins may view all vehicles across all customers.

3.5.2 Vehicle–Subscription Linking (FR-015)

- A vehicle may be linked to a policy subscription at any time.
- Multiple vehicles may be associated with a single subscription.
- The subscription detail view shows all linked vehicles.

3.6 Usage Data Tracking

3.6.1 Monthly Usage Submission (FR-016)

- Customers shall submit monthly driving usage data per subscription.

- Usage data fields: total distance driven (km), night driving hours, trip count, billing month (1–12), and billing year.
- The system automatically derives a risk category (LOW, MEDIUM, HIGH) from the submitted data.
- One usage record per subscription per billing month/year is supported.

3.6.2 Usage History (FR-017)

- Customers may retrieve all historical usage records for a given subscription.
- Individual usage records are retrievable by subscription, year, and month.

3.7 Premium Calculation Engine

3.7.1 Rule-Based Premium Engine (FR-018)

The system employs a Strategy Pattern-based rule engine that computes the final monthly premium:

Rule Type	Example Condition	Effect (Value)	Direction
DISTANCE	> 1500 km	300.00 INR	Addition (+)
DISTANCE	< 500 km	200.00 INR	Discount (-)
NIGHT_DRIVING	> 10 hours	500.00 INR	Addition (+)
RISK_CATEGORY	HIGH	800.00 INR	Addition (+)

- Only rules with isActive=true participate in calculations.
- The engine fetches all active rules from the database and evaluates each against the usage data.
- Final premium = Base Premium + Sum of Applicable Additions - Sum of Applicable Discounts.
- Calculation results (basePremium, totalAdditions, totalDiscounts, finalPremium) are persisted.

3.7.2 Premium Rule Administration (FR-019)

- Admins may create, update, delete, activate, and deactivate premium rules.
- Rule types supported: DISTANCE, NIGHT_DRIVING, RISK_CATEGORY.
- Rule conditions use comparison operators (e.g., '> 1500', '== HIGH').

3.7.3 Premium History (FR-020)

- Customers may retrieve the full history of premium calculations for any subscription.
- Each record captures: base premium, additions, discounts, final premium, and calculation timestamp.

3.8 Claims Management

3.8.1 Claim Submission (FR-021)

- Customers may submit claims against an active policy subscription.
- Claim submission requires: subscription ID, claim amount, claim reason, and supporting documents.
- Accepted document formats: PDF, JPG, JPEG, PNG (max 5 MB each).
- Claim documents are stored at: ./data/claim-documents/{claimId}/
- Claims are created in PENDING status; CLAIMS_OFFICER role users receive an in-app notification.

3.8.2 Claim Review (FR-022)

- Claims Officers may list, search, and filter claims by status.
- Claims Officers may download supporting documents for review.
- Claim decisions: APPROVE or REJECT.
- Upon approval or rejection, the customer receives both an email and an in-app notification.
- The reviewing officer's identity is recorded in the reviewedBy field.

3.8.3 Claim Concurrency Control (FR-023)

> *Race Condition Prevention: The claim approval/rejection uses an atomic SQL UPDATE that matches only on PENDING status. If two officers attempt to process the same claim simultaneously, only one succeeds — the second receives a ClaimAlreadyProcessedException (HTTP 409).*

3.9 Payment Processing

3.9.1 Payment Initiation (FR-024)

- After policy approval, customers may initiate premium payment.
- The system validates that the policy order belongs to the authenticated customer.
- Duplicate payment prevention: if a SUCCESS payment already exists for the order, further payment attempts are rejected with HTTP 400.
- The system creates a Razorpay payment order and returns: orderId, amount (INR), currency, and public key.

3.9.2 Payment Verification (FR-025)

- After Razorpay widget completion, the customer submits: Razorpay Order ID, Payment ID, and HMAC-SHA256 signature.
- The backend validates the signature using the Razorpay key secret.
- On successful verification, payment status is updated to SUCCESS.
- A PDF invoice is generated and emailed to the customer.

3.9.3 Payment History (FR-026)

- Customers may view their own payment history.
- Admins may view payment history for all users.
- Each record shows: transaction ID, policy reference, amount, status (SUCCESS/FAILED/PENDING), and date.

3.10 Notifications

3.10.1 In-App Notifications (FR-027)

Notification Type	Trigger Event	Recipient
NEW_REGISTRATION	Customer self-registers	All ADMINs
STAFF_CREATED	Admin creates a staff account	New staff member
POLICY_APPROVAL	Underwriter approves application	CUSTOMER
POLICY_REJECTION	Underwriter rejects application	CUSTOMER
DOCUMENT_REQUEST	Underwriter requests more documents	CUSTOMER
NEW_CLAIM	Customer submits a claim	CLAIMS_OFFICERs
CLAIM_APPROVAL	Claims officer approves claim	CUSTOMER
CLAIM_REJECTION	Claims officer rejects claim	CUSTOMER
PAYMENT_SUCCESS	Payment verified successfully	CUSTOMER

- Users may retrieve their notifications sorted by newest first.
- Users may mark individual notifications as read.
- Users may delete notifications.

3.10.2 Email Notifications (FR-028)

- Welcome email sent on customer registration.
- Staff portal welcome email on staff account creation.
- Policy approval/rejection/document request emails.
- Claim approval/rejection emails.
- Payment confirmation email with attached PDF invoice.
- Password reset email with a 15-minute TTL link.

3.11 Dashboard & Reporting

3.11.1 Admin Dashboard (FR-029)

- Admin summary provides: total customers, total vehicles, total and active policies, total and pending orders, total and active subscriptions, total/pending/approved claims, and total revenue.
- Statistics dashboard provides: risk distribution (LOW/MEDIUM/HIGH breakdown), monthly revenue trend, claim ratio, active user count, and average premium.

3.11.2 Underwriter Dashboard (FR-030)

- Provides: pending, approved, rejected, and total application counts.
- Highlights high-risk applications count for immediate attention.

4. Data Requirements & Database Design

4.1 Entity Relationship Overview

The DriveIQ database consists of 14 core tables with well-defined foreign key relationships. The following diagram illustrates the primary entity relationships:

```
users ──────────────────── roles
  │
  ├── policy_orders ──────── policies
  │       │                    │
  │       └── policy_subscriptions ── vehicles (via vehicle_subscriptions)
  │               │
  │               ├── usage_data
  │               ├── premium_calculations ── usage_data
  │               └── claims
  │
  ├── payments
  ├── notifications
  └── password_reset_tokens
```

4.2 Core Entity Definitions

4.2.1 Users Table

Column	Type	Constraints	Description
user_id	BIGINT	PK, AUTO_INCREMENT	Unique user identifier
full_name	VARCHAR(255)	NOT NULL	User's full legal name
email	VARCHAR(255)	UNIQUE, NOT NULL	Login credential and contact email
password	VARCHAR(255)	NOT NULL	BCrypt-hashed password
phone	VARCHAR(20)	NULLABLE	Contact phone number
age	INTEGER	NULLABLE	Driver's age for risk scoring
address	VARCHAR(500)	NULLABLE	Residential address
status	VARCHAR(50)	DEFAULT 'ACTIVE'	Account status
role_id	BIGINT	FK → roles	Assigned role
created_at	TIMESTAMP	DEFAULT NOW()	Registration timestamp

4.2.2 PolicyOrder Table

Column	Type	Description
order_id	BIGINT PK	Unique application order identifier
user_id	BIGINT FK	Applying customer
policy_id	BIGINT FK	Selected policy template
vehicle_id	BIGINT FK NULLABLE	Optional linked vehicle
order_status	VARCHAR(50)	PENDING | APPROVED | REJECTED | DOCUMENTS_REQUESTED
order_date	TIMESTAMP	Application submission timestamp
document_names	TEXT (CSV)	Original document filenames
stored_document_names	TEXT (CSV)	UUID-prefixed stored filenames
underwriter_remarks	TEXT	Underwriter decision remarks
risk_score	DOUBLE	Calculated risk score (0–100)
risk_level	VARCHAR(20)	LOW | MEDIUM | HIGH

4.2.3 UsageData Table

Column	Type	Description
usage_id	BIGINT PK	Unique usage record identifier
subscription_id	BIGINT FK	Parent policy subscription
billing_month	INTEGER (1–12)	Reporting month
billing_year	INTEGER	Reporting year
total_distance_km	DOUBLE	Total kilometres driven in billing period
night_driving_hours	DOUBLE	Hours driven between 10 PM–6 AM
trip_count	INTEGER	Number of trips taken
risk_category	VARCHAR(20)	Derived risk: LOW | MEDIUM | HIGH

4.2.4 PremiumRule Table

Column	Type	Description
rule_id	BIGINT PK	Unique rule identifier
rule_type	VARCHAR(50)	DISTANCE | NIGHT_DRIVING | RISK_CATEGORY
condition	VARCHAR(255)	Condition expression (e.g., '> 1500', 'HIGH')
value	DOUBLE	Premium modifier amount (INR) — positive = addition, negative = discount
is_active	BOOLEAN	Whether this rule participates in calculations

4.3 Document Storage Strategy

Document Type	Storage Path	Access Endpoint	Max Size
Policy Documents	./data/policy-documents/{orderId}/	GET /api/policy-orders/{orderId}/documents/{filename}	5 MB
Claim Documents	./data/claim-documents/{claimId}/	GET /api/claims/{claimId}/documents/{filename}	5 MB

- Accepted MIME types: application/pdf, image/jpeg, image/png
- Stored filenames are UUID-prefixed to avoid collisions (e.g., 550e8400-e29b-41d4-a716_original.pdf)
- Download responses include Content-Disposition: attachment header for proper browser handling

5. REST API Specification

5.1 API Standards

- Base URL: http://localhost:8080/api (production URL to be configured per environment)
- All requests and responses use JSON (Content-Type: application/json) unless multipart upload
- Authentication: Bearer token in Authorization header for all non-public endpoints
- Pagination: Standard Spring Page<T> response structure for all list endpoints

5.1.1 Standard Pagination Response

```
{
  "content": [...],
  "pageable": { "pageNumber": 0, "pageSize": 10 },
  "totalElements": 42,
  "totalPages": 5,
  "first": true,
  "last": false,
  "numberOfElements": 10
}
```

5.2 Authentication API

Method	Endpoint	Auth Required	Description
POST	/auth/register	No	Register a new customer account
POST	/auth/login	No	Authenticate and receive JWT token
POST	/auth/forgot-password	No	Request password reset email
POST	/auth/reset-password	No	Reset password using token

5.3 Core Resource APIs

Module	Key Endpoints	Authorized Roles
Policies	GET /policies, POST /policies, PUT /policies/{id}	GET: All | Modify: ADMIN
Policy Orders	POST /policy-orders, GET /policy-orders, GET /policy-orders/user/{id}	CUSTOMER creates; ADMIN/UNDERWRITER views
Underwriter	GET /underwriter/applications, POST /underwriter/approve-policy/{id}	UNDERWRITER only

Module	Key Endpoints	Authorized Roles
Subscriptions	GET /subscriptions, GET /subscriptions/user/{id}, POST /subscriptions/{id}/renew	CUSTOMER own; ADMIN all
Vehicles	POST /vehicles, PUT /vehicles/{id}, DELETE /vehicles/{id}	CUSTOMER owns; ADMIN views all
Claims	POST /claims, PUT /claims/{id}/approve, PUT /claims/{id}/reject	CUSTOMER submits; CLAIMS_OFFICER decides
Usage	POST /usage, GET /usage/subscription/{id}	Authenticated users
Premium	POST /premium/calculate/{subscriptionId}, GET /premium/history/{id}	Authenticated users
Payments	POST /payments/create, POST /payments/verify, GET /payments/history/{id}	CUSTOMER creates; ADMIN views all
Rules	GET /rules, POST /rules, PUT /rules/{id}, DELETE /rules/{id}	ADMIN only
Dashboard	GET /dashboard/summary, GET /statistics/dashboard	ADMIN; Statistics: ADMIN+UNDERWRITER
Notifications	GET /notifications, PUT /notifications/{id}/read, DELETE /notifications/{id}	Own notifications only
Profile	GET /users/profile, PUT /users/profile	All authenticated users
Admin Users	GET /admin/users, PUT /admin/users/{id}/role, PUT /admin/users/{id}/deactivate	ADMIN only

5.4 Payment Flow — Sequence

```
Customer                    Backend                    Razorpay
   |                           |                          |
   | POST /payments/create     |                          |
   |──────────────────────────►|                          |
   |                           | Create Razorpay Order    |
   |                           |──────────────────────────►|
   |                           |◄──────────────────────────|
   |◄──────────────────────────| {orderId, amount, key}   |
   |                           |                          |
   | [Opens Razorpay Widget]   |                          |
   |──────────────────────────────────────────────────────►|
   |◄──────────────────────────────────────────────────────|
   | {razorpayPaymentId, signature}                       |
   |                           |                          |
   | POST /payments/verify     |                          |
```

```
    |──────────────────────────►|                            |
    |                           | HMAC signature validate  |
    |                           | Update payment SUCCESS    |
    |                           | Generate PDF Invoice      |
    |                           | Send Email                |
    |◄──────────────────────────| 200 OK                   |
```

6. Security Requirements

6.1 Authentication & Authorization

6.1.1 JWT-Based Authentication (NFR-001)

- The system uses JWT tokens with RS256 signing algorithm for stateless authentication.
- Token TTL is 24 hours; expired tokens are rejected with HTTP 401.
- The JWT payload contains: user email, role, and user ID.
- Tokens are stored client-side in localStorage under the key 'token'.
- The JwtAuthenticationFilter intercepts all protected requests and validates tokens before any controller executes.
- On 401, the Angular authInterceptor clears localStorage and redirects to /login.

6.1.2 Role-Based Access Control (NFR-002)

- All controller endpoints enforce role-based authorization via Spring Security annotations (@PreAuthorize).
- Public endpoints are limited to: /api/auth/** and /error.
- Role mismatch returns HTTP 403; missing token returns HTTP 401.
- Frontend route guards mirror backend security: authGuard validates role against stored JWT claims.

6.1.3 Password Security (NFR-003)

- All passwords are hashed using BCrypt before storage. Plain-text passwords are never persisted.
- Password reset tokens are single-use and expire after 15 minutes.
- Token uniqueness is enforced via database UNIQUE constraint.

6.2 Data Security

6.2.1 Payment Security (NFR-004)

- Razorpay HMAC-SHA256 signature verification is mandatory before recording payment success.
- The Razorpay secret key is stored as an application property and never exposed in API responses.
- Duplicate payment prevention uses a database-level existence check before creating any Razorpay order.

6.2.2 Input Validation (NFR-005)

- All incoming DTOs are validated using Jakarta Validation (Hibernate Validator).
- File uploads are restricted to allowed MIME types (PDF, JPG, JPEG, PNG) and 5 MB size limit.
- Email format, non-null constraints, and minimum value checks are enforced server-side.

6.3 Controller Security Reference

Controller	URL Pattern	Security Rule
AuthController	/api/auth/**	Public — no authentication required
UserManagementController	/api/admin/users/**	@PreAuthorize("hasRole('ADMIN')")
UserProfileController	/api/users/profile	Any authenticated user (JWT required)
PolicyController	/api/policies/**	GET: All authenticated | POST/PUT: ADMIN only
UnderwriterController	/api/underwriter/**	@PreAuthorize("hasRole('UNDERWRITER')")
ClaimController	/api/claims (GET/action)	@PreAuthorize("hasAnyRole('ADMIN','CLAIMS_OFFICER')"
ClaimController	/api/claims (POST)	CUSTOMER role
PaymentController	/api/payments/create,verify	@PreAuthorize("hasRole('CUSTOMER')")
PaymentController	/api/payments (GET all)	@PreAuthorize("hasRole('ADMIN')")
RuleController	/api/rules/**	ADMIN only
DashboardController	/api/dashboard/**	ADMIN only
StatisticsController	/api/statistics/**	ADMIN + UNDERWRITER

7. Non-Functional Requirements

7.1 Performance Requirements

Requirement ID	Category	Requirement
NFR-006	Response Time	API responses for standard read operations shall complete within 500 ms under normal load
NFR-007	Pagination	All list endpoints support pagination (default 10 items/page) to limit response payload size
NFR-008	File Upload	Document uploads are validated and stored within 3 seconds for files up to 5 MB
NFR-009	Premium Calculation	Premium rule engine evaluation shall complete within 200 ms for any single calculation

7.2 Scalability Requirements

Requirement ID	Category	Requirement
NFR-010	Stateless Backend	The Spring Boot backend is fully stateless (session-less). Horizontal scaling is supported by running multiple backend instances behind a load balancer
NFR-011	Database	PostgreSQL connection pooling is used to handle concurrent requests. The schema supports JPA lazy loading to minimize unnecessary data fetching
NFR-012	Frontend	Angular SPA uses lazy-loaded routes to minimize initial bundle size and improve load time

7.3 Reliability Requirements

Requirement ID	Category	Requirement
NFR-013	Financial Transactions	All payment and claim operations use atomic database transactions to prevent partial state corruption
NFR-014	Concurrency	Claim approval uses optimistic concurrency control (atomic SQL UPDATE on PENDING status) to prevent double-processing
NFR-015	Payment Idempotency	Duplicate payment creation is prevented by existence check before any Razorpay API call
NFR-016	Audit Trail	Risk scores, underwriter decisions, and payment records are immutably persisted for auditing

7.4 Usability Requirements

- The Angular SPA provides a responsive single-page interface requiring no page reloads for standard navigation.
- Role-specific dashboards ensure each user sees only the functionality relevant to their role.
- All system events (approvals, rejections, payments) trigger immediate in-app notifications visible without page refresh.
- Email confirmations are sent for all major lifecycle events providing an offline record.

7.5 Maintainability Requirements

- Premium rules are database-driven and configurable by Admin at runtime without code changes.
- The Premium Rule Engine uses the Strategy Pattern, allowing new rule types to be added by implementing a new strategy class.
- Environment-specific configuration (database URLs, API keys, SMTP credentials) is externalized via application.properties.
- All repository queries use JPQL with named parameters for readability and SQL injection prevention.

7.6 Security Compliance Requirements

Requirement ID	Standard	Requirement
NFR-017	OWASP	Passwords are BCrypt-hashed with adequate cost factor. Never stored in plain text
NFR-018	JWT RFC 7519	JWT tokens include expiry claim; tokens are validated on every protected request
NFR-019	PCI-DSS (Partial)	Card data is handled entirely by Razorpay; DriveIQ never stores or transmits raw card data
NFR-020	HTTPS	All production traffic shall be served over HTTPS (TLS 1.2+)

8. System Architecture & Technology Stack

8.1 Backend Technology Stack

Layer	Technology	Version / Details
Language	Java	17+ LTS
Framework	Spring Boot	3.x
Security	Spring Security + JWT	RS256 algorithm, 24-hour TTL
Database ORM	JPA / Hibernate	With Spring Data JPA repositories
Database	PostgreSQL	14+ (DDL auto=update)
Build Tool	Maven	Standard POM structure
Email	JavaMail / SMTP	Gmail SMTP, port 587, TLS
Payments	Razorpay Java SDK	HMAC-SHA256 payment verification
PDF Generation	iText / Custom PDF Generator	For invoice generation
Validation	Jakarta Validation	Hibernate Validator implementation
Password Hashing	BCrypt	Via Spring Security PasswordEncoder

8.2 Frontend Technology Stack

Layer	Technology	Details
Language	TypeScript	Strongly typed Angular components
Framework	Angular (Standalone Components)	Modern Angular architecture without NgModules
Styling	Tailwind CSS + Custom CSS	Utility-first styling with custom overrides
State Management	RxJS + BehaviorSubject	Reactive streams for asynchronous state
HTTP Client	Angular HttpClient	With authInterceptor for JWT injection
Authentication	JWT (localStorage)	Token stored under key 'token'
Route Protection	authGuard	Role-based route access control

8.3 Frontend Module Structure

```
src/app/
├── core/
│   ├── guards/          authGuard (role-based route protection)
│   ├── interceptors/    authInterceptor (JWT header injection)
│   ├── models/          models.ts (DTOs: User, Policy, Subscription, etc.)
```

```
│   └── services/       user.service.ts
├── features/
│   ├── auth/           Login | Register | Forgot-Password | Reset-Password
│   ├── admin/          Dashboard | Policies | Orders | Users | Vehicles
│   │                   Claims | Rules | Statistics
│   ├── customer/       Dashboard | Policies | Vehicles | Subscriptions
│   │                   Usage | Claims | Premium | Payments
│   ├── underwriter/    Dashboard | Orders (Applications) | Review
│   │                   Policies | Statistics
│   ├── claims-officer/ Dashboard | Pending Claims | Claims History
│   └── profile/        View/Edit Profile
└── shared/
    ├── components/     Navbar | Notifications
    └── services/       CoverageService | OrderService | PolicyService
                        PremiumService | SubscriptionService
```

8.4 Angular Route Protection

All privileged routes are guarded by authGuard, which enforces the following logic:

8. Check if localStorage['token'] exists → if not, redirect to /login.
9. Decode JWT and extract role claim.
10. Compare extracted role against route's required role (route.data.role).
11. If mismatch → redirect to role-appropriate dashboard.

Route	Required Role
/admin/dashboard	ADMIN
/admin/users	ADMIN
/customer/dashboard	CUSTOMER
/customer/payments	CUSTOMER
/underwriter/dashboard	UNDERWRITER
/underwriter/orders	UNDERWRITER
/claims/dashboard	CLAIMS_OFFICER
/claims/manage	CLAIMS_OFFICER
/profile	Any Authenticated User

9. Business Rules & State Machines

9.1 Core Business Rules

BR-001: Single Active Policy

A customer can apply for multiple policies, but only one APPROVED policy order creates an active subscription at any given time. Multiple active subscriptions are not permitted for the same customer.

BR-002: Document Validation

Policy application and claim document uploads are restricted to the following formats: PDF, JPG, JPEG, PNG. Maximum file size per document is 5 MB. Files outside these specifications are rejected with HTTP 400.

BR-003: Atomic Claim Processing

Claims processing is concurrency-safe. An already-processed claim (non-PENDING status) cannot be re-approved or re-rejected. The system uses an atomic SQL UPDATE with a PENDING status precondition to prevent race conditions.

BR-004: Payment Idempotency

Once a payment record with status SUCCESS exists for a given policy order, any subsequent payment creation attempt for the same order is rejected with HTTP 400 ("Premium already paid.").

BR-005: Risk Score Locking

Risk is calculated dynamically while an application is in PENDING status. At the moment of underwriter decision (Approve or Reject), the risk score and risk level are locked and persisted in the policy order record for audit trail purposes.

BR-006: Premium Rule Activation

Only premium rules with isActive=true participate in premium calculations. Inactive rules are ignored by the rule engine. Rules can be toggled by Admins at any time, taking effect on the next calculation.

BR-007: Subscription Duration

All policy subscriptions have a fixed 1-year duration from the approval date. startDate equals the approval date; endDate equals startDate plus 365 days.

BR-008: Profile Immutability

A user's email address is immutable after registration and cannot be changed through any profile update endpoint.

9.2 State Machine Diagrams

9.2.1 PolicyOrder Status Flow

```
                    ┌──────────┐
                    │ PENDING  │
                    └──────────┘
                         │
          ┌──────────────┼──────────────┐
          │              │              │
          ▼              ▼              ▼
     ┌──────────┐   ┌──────────┐   ┌──────────────────┐
     │ APPROVED │   │ REJECTED │   │ DOCUMENTS_REQUESTED │
     └──────────┘   └──────────┘   └──────────────────┘
                                            │
                                            │
                                  (Customer resubmits)
                                            │
                                            ▼
                                       ┌──────────┐
                                       │ PENDING  │
                                       └──────────┘
```

9.2.2 Claim Status Flow

```
     ┌──────────┐
     │ PENDING  │
     └──────────┘
          │
     ┌────┴─────┐
     │          │
     ▼          ▼
┌──────────┐ ┌──────────┐
│ APPROVED │ │ REJECTED │
└──────────┘ └──────────┘
```

9.2.3 PolicySubscription Status Flow

```
     ┌────────┐                    ┌────────┐
     │ ACTIVE │──────────────────► │ EXPIRED │
     └────────┘                    └────────┘
         │                             │
         │ CANCEL                      │
         │                        RENEW│
         ▼                             ▼
     ┌──────────┐                 ┌────────┐
     │ CANCELLED │                │ ACTIVE │
     └──────────┘                 └────────┘
```

9.2.4 Payment Status Flow

```
     ┌────────┐
     │ PENDING │
     └────────┘
          │
```

```
        ┌────┴────┐
        │         │
        ▼         ▼
   ┌─────────┐ ┌────────┐
   │ SUCCESS │ │ FAILED │
   └─────────┘ └────────┘
```

10. End-to-End Workflows

10.1 Policy Application to Active Subscription

This workflow covers the complete lifecycle from a customer selecting a policy through to an active, paid subscription.

Step	Actor	Action	System Response
1	Customer	Browse available policies	Returns paginated list of active policies
2	Customer	Select policy + vehicle + upload documents	Creates PolicyOrder (PENDING); stores documents
3	Underwriter	View pending applications with risk scores	Returns paginated list with computed risk data
4A	Underwriter	Approve application with remarks	Status→APPROVED; PolicySubscription created; customer notified
4B	Underwriter	Request additional documents	Status→DOCUMENTS_REQUESTED; customer emailed
4C	Underwriter	Reject application	Status→REJECTED; customer notified
5	Customer	Initiate premium payment	Razorpay order created; payment widget displayed
6	Customer	Complete payment in Razorpay widget	HMAC verified; payment SUCCESS; PDF invoice emailed
7	Customer	View active subscription	subscriptionStatus: ACTIVE; usage tracking available

10.2 Claim Submission to Settlement

Step	Actor	Action	System Response
1	Customer	Identify active subscription	Returns list of active subscriptions
2	Customer	Submit claim with supporting documents	Claim created (PENDING); Claims Officers notified
3	Claims Officer	View and review pending claims	Returns paginated list of pending claims
4A	Claims Officer	Approve claim	Status→APPROVED; customer notified by email + in-app
4B	Claims Officer	Reject claim	Status→REJECTED; customer notified by email + in-app

10.3 Monthly Usage and Premium Calculation

Step	Actor	Action	System Response
1	Customer	Submit monthly usage data (distance, night hours, trip count)	Usage record created; risk category derived
2	Customer	Request premium calculation for the month	Rule engine evaluates all active rules against usage data
3	System	Apply matching ADDITION rules	Sum of additions computed
4	System	Apply matching DISCOUNT rules	Sum of discounts computed
5	System	Persist calculation result	PremiumCalculation record stored; final premium returned
6	Customer	View premium history	Returns all historical calculations for subscription

11. Error Handling & API Error Reference

11.1 Standard HTTP Error Codes

HTTP Status	Exception Class	Trigger Condition
400 Bad Request	BadRequestException	Invalid input data, business rule violation, duplicate payment attempt, or invalid file type
401 Unauthorized	Spring Security (auto)	Missing or expired JWT token in Authorization header
403 Forbidden	Spring Security (auto)	Valid token but insufficient role permissions for the requested resource
404 Not Found	ResourceNotFoundException	Requested entity (user, policy, claim, subscription) does not exist in the database
409 Conflict	ClaimAlreadyProcessedException	Concurrency conflict: claim was already approved or rejected by another officer simultaneously

11.2 Error Response Format

```
{
  "timestamp": "2026-03-17T10:00:00.000Z",
  "status": 404,
  "error": "Not Found",
  "message": "Policy order not found with id: 99",
  "path": "/api/policy-orders/99"
}
```

11.3 Concurrency Control Implementation

The following code pattern illustrates the atomic claim processing mechanism that prevents race conditions:

```
// ClaimRepository.java
@Modifying
@Query("UPDATE Claim c SET c.claimStatus = :status, c.reviewedBy = :reviewer "
       + "WHERE c.claimId = :claimId AND c.claimStatus = 'PENDING'")
int updateClaimStatus(@Param("claimId") Long claimId,
                      @Param("status") String status,
                      @Param("reviewer") User reviewer);

// ClaimsService.java
int rowsUpdated = claimRepository.updateClaimStatus(claimId, status, officer);
if (rowsUpdated == 0) {
```

```
    throw new ClaimAlreadyProcessedException("Claim already processed.");
}
```

12. Configuration & Deployment

12.1 Environment Configuration

All environment-specific values are managed via application.properties. The following properties must be configured before deployment:

Property	Description	Example Value
server.port	Backend server port	8080
spring.datasource.url	PostgreSQL connection URL	jdbc:postgresql://localhost:5432/driveiq
spring.jpa.hibernate.ddl-auto	Schema management strategy	update
jwt.secret	JWT signing secret key	<64-char random string>
jwt.expiration	JWT token TTL in milliseconds	86400000 (24 hours)
razorpay.key.id	Razorpay public API key	rzp_test_LzQZPQlKJQ8G9G
razorpay.key.secret	Razorpay secret for HMAC verification	<secret>
spring.mail.host	SMTP server host	smtp.gmail.com
spring.mail.port	SMTP server port	587
claims.upload-dir	Claims document storage path	./data/claim-documents
policy.upload-dir	Policy document storage path	./data/policy-documents

12.2 Deployment Prerequisites

- Java Development Kit (JDK) 17 or later installed on the server.
- PostgreSQL 14+ database server running and accessible.
- Database 'driveiq' created with appropriate user permissions.
- Node.js 18+ and Angular CLI installed for frontend build.
- Valid Razorpay account with API keys for the target environment (test/live).
- Gmail account configured with App Password for SMTP relay.
- ./data/claim-documents and ./data/policy-documents directories created with write permissions.

12.3 Build & Run Commands

```
# Backend
mvn clean install
java -jar target/driveiq-backend.jar
```

```
# Frontend
npm install
ng build --configuration=production
ng serve   # Development only
```

13. Data Transfer Object (DTO) Reference

13.1 Authentication DTOs

DTO Name	Key Fields	Validation Rules
RegisterRequest	fullName, email, password, phone, age	fullName NotBlank; email valid Email format; password NotBlank
LoginRequest	email, password	Both required
ForgotPasswordRequest	email	Valid Email, NotBlank
ResetPasswordRequest	token, newPassword	Both NotBlank
AuthResponse	token, email, role, userId	Returned on successful login
RegisterResponse	message, email, userId	Returned on successful registration

13.2 Domain DTOs

DTO Name	Key Fields	Notes
ClaimRequest	subscriptionId, claimAmount (min=1), claimReason	Multipart — includes file array
ClaimResponse	claimId, claimAmount, claimStatus, submittedDate, reviewedBy, documentNames	reviewedBy is nullable
PolicyRequest	policyName, coverageType, basePremium, description, isActive	COMPREHENSIVE or THIRD_PARTY
UnderwriterApplicationResponse	orderId, customerName, riskScore, riskLevel, vehicleType, nightDrivingHours, totalDistanceKm	Full application view
UnderwriterDecisionRequest	remarks	Required for approve/reject/request-docs
PaymentOrderResponse	orderId, amount, currency (INR), key	Used to initialize Razorpay widget
PaymentVerifyRequest	razorpayOrderId, razorpayPaymentId, razorpaySignature	HMAC-SHA256 signature required
UsageRequest	subscriptionId, billingMonth (1-12), billingYear, totalDistanceKm,	One record per month/year/subscription

DTO Name	Key Fields	Notes
	nightDrivingHours, tripCount	
PremiumCalculationResponse	basePremium, totalAdditions, totalDiscounts, finalPremium, calculatedDate	Result of rule engine execution
PremiumRule	ruleId, ruleType, condition, value, isActive	DISTANCE / NIGHT_DRIVING / RISK_CATEGORY

14. Appendices

Appendix A: Premium Rule Engine — Strategy Pattern

The Premium Rule Engine is implemented using the Strategy design pattern, enabling extensible, plug-in rule evaluation without modifying core business logic.

```
Rule Strategy Pattern:

interface PremiumRuleStrategy {
    boolean applies(UsageData usage, PremiumRule rule);
}

DistanceRuleStrategy     → evaluates UsageData.totalDistanceKm against condition
NightDrivingRuleStrategy → evaluates UsageData.nightDrivingHours against condition
RiskCategoryRuleStrategy → evaluates UsageData.riskCategory against condition

Engine Logic:
  1. Fetch all active PremiumRules from database
  2. For each rule, instantiate the appropriate strategy
  3. Evaluate strategy.applies(usageData, rule) → boolean
  4. If true: add rule.value to additions or discounts
  5. Result: { basePremium, totalAdditions, totalDiscounts, finalPremium }

Sample Evaluation:
  distanceKm = 1800 → rule '>1500, +300' applies → additions += 300
  nightHours = 15   → rule '>10, +500' applies  → additions += 500
  riskCategory = LOW → no HIGH rule applies     → no addition
  basePremium = 5200
  finalPremium = 5200 + 300 + 500 = 6000
```

Appendix B: Risk Score Computation

Risk scoring is performed dynamically during the underwriting review phase. The score (0–100) is computed based on the following factors:

- Driver age — younger drivers (under 25) attract higher risk scores
- Vehicle type — SPORTS and TRUCK types carry higher base risk
- Vehicle age — older vehicles contribute to higher risk
- Claims history — prior approved claims increase risk score
- Night driving hours — if recent usage data is available, high night driving hours elevate risk

Risk levels are assigned based on the computed score: 0–40 = LOW, 41–70 = MEDIUM, 71–100 = HIGH.

Appendix C: Sample cURL API Commands

C.1 Register a New Customer

```
curl -X POST http://localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "fullName": "Jane Doe",
    "email": "jane@example.com",
    "password": "Jane@1234",
    "phone": "9876500000",
    "age": 29
  }'
```

C.2 Submit Monthly Usage Data

```
curl -X POST http://localhost:8080/api/usage \
  -H "Authorization: Bearer <JWT_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{
    "subscriptionId": 12,
    "billingMonth": 3,
    "billingYear": 2026,
    "totalDistanceKm": 790.5,
    "nightDrivingHours": 8.0,
    "tripCount": 55
  }'
```

C.3 Approve a Policy Application

```
curl -X POST http://localhost:8080/api/underwriter/approve-policy/22 \
  -H "Authorization: Bearer <UNDERWRITER_JWT_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"remarks": "All documents verified. Approved."}'
```

C.4 Calculate Premium

```
curl -X POST "http://localhost:8080/api/premium/calculate/12?usageId=7" \
  -H "Authorization: Bearer <JWT_TOKEN>"
```

Appendix D: Notification Type Quick Reference

Type	Sender	Recipient	Email Also Sent?
NEW_REGISTRATION	System (auto)	All ADMINs	Yes — Welcome email to customer

Type	Sender	Recipient	Email Also Sent?
STAFF_CREATED	Admin	New staff member	Yes — Staff portal welcome
POLICY_APPROVAL	Underwriter	Customer	Yes
POLICY_REJECTION	Underwriter	Customer	Yes
DOCUMENT_REQUEST	Underwriter	Customer	Yes
NEW_CLAIM	Customer	All CLAIMS_OFFICERs	No
CLAIM_APPROVAL	Claims Officer	Customer	Yes
CLAIM_REJECTION	Claims Officer	Customer	Yes
PAYMENT_SUCCESS	System (auto)	Customer	Yes — with PDF Invoice

Appendix E: Revision History

Version	Date	Author	Description
1.0	March 17, 2026	Project Team	Initial release — complete SRS for DriveIQ UBI Platform v1.0

End of Document

DriveIQ Insurance — Software Requirements Specification v1.0 | March 2026